

Converting a PDF textbook to be accessible

Timothy Prescott

Abstract

A guideline in the United States requires that PDFs created after April 2026 must be accessible to visually impaired users. We discuss how we updated a custom three semester calculus book to be accessible, including what we did, the challenges we faced, how we verified accessibility, and what remains to be done. We also discuss necessary adaptations to satisfy a common accessibility checker available in several LMSs.

1 Introduction & Setup

Since 2016, the University of North Dakota has used a custom 1150 page textbook for its three semester calculus sequence. The source for the textbook is at github.com/teepeeemm/APEXCalculusLT_Source, forked from github.com/APEXCalculus/APEXCalculus_Source while the compiled document is at arts-sciences.und.edu/academics/math/calc-1-texts.html.

Using a custom textbook has enabled students to download the PDF for free in addition to having the option to purchase the book for a fraction of the cost of other textbooks (currently \$25–\$30 per semester, when the author last checked).

The format of the book is to `\include` one `\part` for each semester, so that `\includeonly` allows us to have a one semester PDFs that we send to the university printer to create physical copies (the online PDF includes all three semesters). Within each included file, we have a few paragraphs introducing each chapter and then `\input` each section. Each section then ends with a custom command that inputs the exercises for that section.

Our university has been emphasizing the national requirement that all documents must be accessible according to the WCAG 2.1 AA guideline as of April 2026 (now delayed to 2027), see *TUGboat* 45:2 [1] for a historical overview of how we got to this point. Beginning in early 2025, we began exploring how to ensure that the textbook meets this guideline.

2 The Basics

We first needed to convert the document from using $\text{\LaTeX}$ to $\text{\LuaTeX}$, which also necessitated adjusting a few font commands in the preamble, but not much else.

The $\text{\LaTeX}$ team has been reporting their progress developing tagging since at least *TUGboat* 40:2 [2] (see also tug.org/TUGboat/Contents/listkeyword.html#CatTAGAccessibility), building off of exist-

ing efforts. Thanks to their hard work, the vast majority of accessibility is enabled by using

```
\DocumentMetadata{
  lang=en,
  tagging=on,
  pdfstandard=ua-2,
% check-tagging-status,
  tagging-setup={
%   math/alt/use,
    math/setup=mathml-SE } }
\RequirePackage{unicode-math}
```

as the first command of the document (even before the `\documentclass`). We'll discuss the commented options as we proceed; other options are available by looking through `texdoc documentmetadata-support-code`. In addition to enabling tagging, the `\DocumentMetadata` command also allows us to adjust the PDF metadata of the compiled document.

An all too common occurrence during compilation was a tagging related error, often that “The number of automatic begin and end hooks differ!” Our setup of using `\include` and `\input` allowed us to comment out various sections, isolating a single section that caused the error. We then examined the log file, found a “Package tagpdf Warning” regarding some aspect of our code, and created a minimal working example causing this warning and error.

With the example in hand, we began searching the internet for a fix. For packages, a good starting point was to view latex3.github.io/tagging-project/tagging-status/, which reports the tagging compatibility of each class and package. Often, a linked issue would contain code snippets we could copy and paste into the preamble that would resolve the issue. We can also create a tagging status report in the log file by uncommenting the `check-tagging-status` line in our `\DocumentMetadata`.

The `tagging-status` website and log information have five categories:

compatible Everything should work for the package, and the $\text{\LaTeX}$ team wants to know if it doesn't. This is the most common entry for the website.

partially-compatible The package should work if you avoid issues described in the website.

currently-incompatible Double check the issue(s) in the website to see if you can make it work.

no-support / unsupported You need to find a new package. (As an aside, if a required package in this category is causing errors because you use `\DocumentMetadata` solely to adjust the PDF metadata, you can use the `pdfmanagement` package instead of `\DocumentMetadata`.)

unchecked / unknown / unclassified The package has not been tested. Your odds are good if some other package loaded it; your odds are bad if it's a local package provided by a "friend".

Resolving errors not directly related to packages was more difficult. We often turned to `tex.stackexchange.com/` for help. Luckily for the reader (and the author), some errors were caused by a bug in the tagging code that was quickly fixed. (A quickly fixed bug was also occasionally true of errors relating to packages.)

3 Alt Text

With the bulk of the work handled automatically by Lua $\text{\LaTeX}$, the greatest part of the work requiring manual intervention was providing alternative text for the thousand images throughout the book. Alt text provides a verbal description of an image that a visually impaired person has difficulty seeing. Ideally, alt text should be

- brief (at most 120 characters or 2 sentences)
- objective
- not the sole source of necessary information (most users will not see the alt text)
- not begin with the word "image", so that a screen reader does not say "Image: image ..."
- not blindly copied from an AI output

$\text{\LaTeX}$'s tagging code allows us to specify alt text as an optional argument to the image:

- `\includegraphics[alt={...}]`
- `\tikz[alt={...}]`
- `\begin{tikzpicture}[alt={...}]`
- `\begin{picture}[alt={...}]`

We also have two other options to `alt={...}`: if the image is purely decorative and unnecessary for the reader, we can use the option `artifact` so that screen readers ignore it entirely. Less commonly, we can use `actualtext={...}` so that screen readers act as if there is no image, and instead read the text. This works best if the image is actually stylistically decorated text.

It is useful to note that it is not too difficult to create alt text. A majority of our discussion requires some familiarity with creating a longer $\text{\LaTeX}$ document. For alt text, however, we used a short Python script that added `[alt=ALT-TEXT-TO-BE-DETERMINED]` to all of the images. A student worker with less $\text{\TeX}$ experience was then able to go through the book, examine the resulting PDF, and supply the alt text.

There are two contrasting problems arising from alt text. The first was succinctly stated by a colleague: "What alt text would you give for the snake

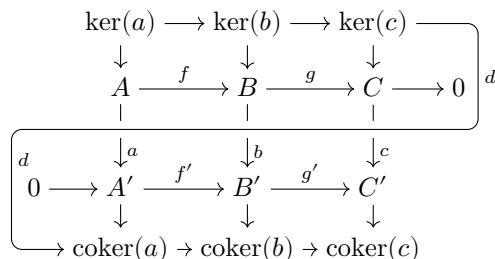


Figure 1: The conclusion of the snake lemma. Well, how would you write alt text for this? Image code adapted from `tex.stackexchange.com/a/440352/`.

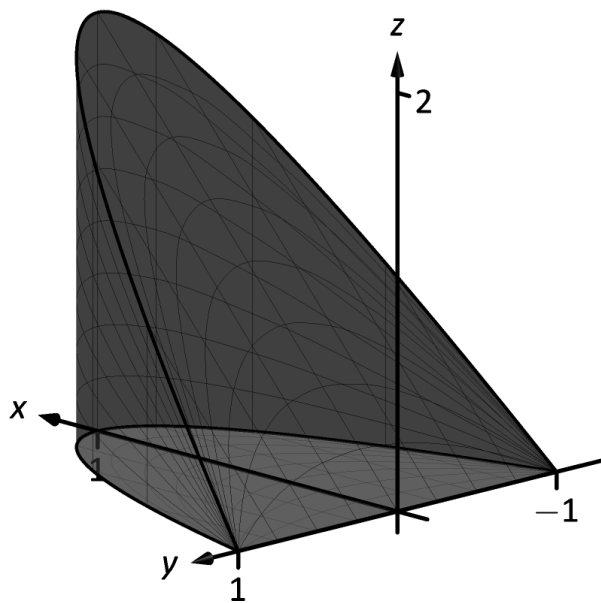


Figure 2: A figure from the textbook with alt text: A wedge where $0 < z < 2x$ and $0 < x < 1 - y^2$. Make sure to grab and rotate the image.

lemma?" (see Figure 1). More pertinent to our discussion, a defining feature of the textbook is its inclusion of numerous three dimensional images created with Asymptote that the reader is able to rotate in the PDF with appropriate viewers (see Figure 2 for an example). It is not reasonable to describe a commutative diagram with fourteen cells in at most 120 characters, nor can a short description convey the information learned by rotating an image. On the other hand, there is no *mathematical* problem with having a longer description. Accessibility guidelines suggest creating a short alt text pointing to a longer description elsewhere.

A bigger problem arises in early chapters in the calculus sequence when we ask students to verbally describe a graph, as in Figure 3. We chose to describe the graph using various synonyms ("the graph is a line from $(-2, 0.5)$ to a hollow dot at $(-1, 1)$, ..."),

Exercise: Describe this function, identifying limiting values, intercepts and their degree, critical and inflection points, and regions where the function is increasing, decreasing, and concave up or down.

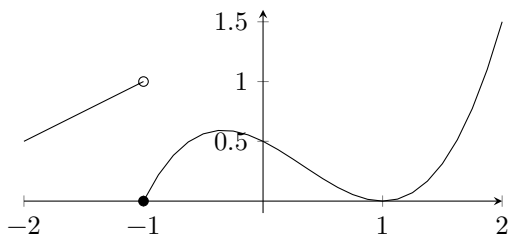


Figure 3: How do you write alt text for this exercise without giving away the answer?

but acknowledge this is not an entirely satisfactory approach.

3.1 Other Issues Related to Alt Text

Many PDF checkers do not recognize PDF version 2.0 and instead check the PDF against version 1.7. A critical distinction for us is that PDF version 1.7 requires that formulas include alt text. We can accomplish this by uncommenting the `math/alt/use` key in the `\DocumentMetadata` command. This usually causes PDF screen readers to read the alt text and omit the more informative MathML tags, so we discourage its use unless a document is being judged on its adherence to the 1.7 standard.

A colleague uses the `rmarkdown` package, which has its own syntax for including graphics. This does not (yet?) allow us to specify the alt text, which we therefore accomplish by using `\setkeys{Gin}{alt={...}}`. Note that this setting will remain in effect until the next such command or when the group ends.

4 Tables

Tables need special consideration so that accessibility tools are able to identify the headings that describe the data in the table. But first, it should be emphasized that if the sole purpose of the table is to arrange elements in a grid, then it's not actually a table and probably shouldn't be tagged as such (there are ways to tell the tagging code "this doesn't need to be tagged as a table", but life will probably be easier if it's not a table to begin with).

To identify the headings in a particular tabular, we use

```
\tagpdfsetup{
  table/header-rows={...},
  table/header-columns={...} }
```

The default is an empty list specifying no header along that axis, so that the most common occurrence of a single header row only needs

```
\tagpdfsetup{table-header-rows={1}}
```

and can omit the `table-header-columns`. These declarations obey TeX's grouping rules, so that a header specification will last until the end of the current environment.

While `\multicolumn` is tagging ready, we cannot say the same for a `\multirow` cell, which needs to have

```
\tagpdfsetup{table/multirow={\langle number of rows \rangle}}
```

at the beginning of the cell.

We should also note that the table interface will likely change over the next few years. Follow `texdoc latex-lab-table` for the details.

5 Other Package Changes

We had originally formatted our headings using the `titlesec` package. This requires adjustments to properly tag, and was becoming too complicated for our use. We opted to instead use `\@startsec` (see `texdoc latex2e`, section 6.8). Another possibility is the new template mechanism (see `texdoc latex-lab-sec-template`).

We were originally using `enumitem` to adjust our `{enumerate}` environments, which is also not amenable to tagging. We are now using `enumext`, although there is also a templating alternative (see `texdoc blocks-doc`).

We also made heavy use of the `marginnote` package which, along with `marginfit`, is currently incompatible with tagging. Instead, we are now using `marginalia`.

6 Checking the Results

Having resolved all possible errors, we needed to verify that our result was indeed accessible. The obvious first step is to ensure a screen reader properly handles the document. Unfortunately, when it comes to PDF version 2.0, the only currently viable PDF readers are Adobe, Foxit, and Firefox, while the only currently viable screen readers are to go with them are NVDA and JAWS on Windows and Orca on Linux. See tex.stackexchange.com/a/755945/ for the latest.

Another approach is to verify that our PDF is indeed version 2.0, and satisfies the UA-2 PDF standard. Unfortunately (again), there are not many PDF checkers that recognize version 2.0, and many will apply version 1.7 standards to a version 2.0 document, resulting in many claimed "issues" with a valid document. pdfa.org/accessible-math-in-pdf-finally/ lists several (but currently only five) checkers that support PDF version 2.0. We have experience with veraPDF (verapdf.org/) and PDFfix (pdfix.net/).

Table 1: Successful validations examined

Profile	Rules	Checks
PDF/UA-2	1723	1292 6774
WTPDF 1.0 Accessibility	1719	1292 6770
WTPDF 1.0 Reuse	1706	1167 9799

The L^AT_EX team uses veraPDF to validate their documents, so that using the same validation gives similar results (this is usually good, but can be bad if they have overlooked an issue). The main downside we found with veraPDF is that while it provides a GUI to initiate a validation and see the results, it reports the location of failed validation checks as paths within the PDF document, such as

```
root/
  document[0]/
    pages[14](925 0 obj PDPAGE)/
      contentStream[0](926 0 obj
        PDSemanticContentStream)/
        content[124]{mcid:109}/
          contentItem[0]{mcid:109}
```

VeraPDF also has a command line version of the program, allowing for faster (or even automated) checking of generated PDFs. After a minute examining the Calculus textbook, veraPDF presents an XML report that the text book complies with various profiles and a number of rules and checks for each profile, summarized in Table 1.

PDFix uses a similar checker as veraPDF, but it also provides a graphical view of the PDF in question, which better locates errors. PDFix also allows navigating the tree structure of the PDF, making it possible to locate errors given by veraPDF.

Our institution uses Ally to check PDFs posted to our LMS. Ally is one of the aforementioned PDF checkers that looks for version 1.7 and requires formulas to have alt text. For this reason, we try to avoid posting the textbook to the LMS.

Ally also requires that documents over two pages use headings. We normally accomplish this with `\section` commands, but if a document does not otherwise have them, it is possible to have the title also provide a heading, see tex.stackexchange.com/a/758805/.

For shorter documents, we can also examine the tagging structure of a PDF by using the command line tool `show-pdf-tags` or the website ngpdf.com/.

Accessible PDFs require more computing power to create. In Table 2, we report the factor by which various computational statistics increase for the textbook. The final two entries relate how much memory the compilation process allocates. This originally resulted in fatal compilation errors that T_EX exceeded

Table 2: Factor by which various statistics increase for compiling the tagged textbook.

Statistic	Factor
Time	6.7
RAM	4.4
Output size	1.3
strings	11
hash	11

its capacity for the **number of strings** or the **hash size**, requiring us to modify the compilation command (see tex.stackexchange.com/a/741777/).

References

- [1] J. Kuan. Legal and cultural landscape of mathematics accessibility in the United States: 2024. *TUGboat* 45(2):189–192, 2024. doi.org/10.47397/tb/45-2/tb140kuan-accessibility
- [2] C. Rowley, U. Fischer, F. Mittelbach. Accessibility in the L^AT_EX kernel — experiments in Tagged PDF. *TUGboat* 40(2):157–158, 2019. tug.org/TUGboat/tb40-2/tb125rowley-tagpdf.pdf

◇ Timothy Prescott
University of North Dakota
101 Cornell Street Stop 8376
Grand Forks ND, 58203
USA
timothy dot prescott (at) UND
dot edu
ORCID 0009-0009-0555-8074